

Chapter 6

Transformers

Exam Study Guide, Worked Concepts, Sample Questions, and

Prepared from	uploaded course materials
Main source	Chapter_6_Transformers.ipynb
Exam source	Chapter_6_Summary_and_Sample_Questions.pdf
Practice source	Chapter_6_Flashcards.pdf

How to use it

Study the probabilistic sequence objective first, then the architecture. Next-token prediction is both a likelihood factorization and the training signal for GPT-style models.

Study order: First read the concept sections, then solve the five sample exam questions, then test yourself with the flashcards.

Contents

- 1. Quick exam map
- 2. Autoregressive modeling
- 3. RNNs, LSTMs, and BPTT
- 4. Self-attention: queries, keys, values
- 5. Multi-head attention and Transformer blocks
- 6. Positions and causal masking
- 7. Tokenization and GPT training
- 8. Formula cheat sheet and symbol key
- 9. Homework and code prompts
- 10. Exam-style questions
- 11. Flashcards

Most useful mental shortcut

Transformers replace the recurrent bottleneck with content-based retrieval: queries ask, keys match, values provide content.

1. Quick exam map

Transformers model sequences by letting tokens directly interact through attention instead of compressing all context into one recurrent hidden state.

Text

token sequence Embedding + position token meaning plus order

Causal

self-attention look only at past

tokens

Transformer

blocks mix and transform context

Next-token

distribution predict what comes next Figure: the main method pipeline for this chapter.

Exam topic	What you must be able to say
Autoregressive	factorization $p(x_{1:T}) = \prod_t p(x_t x_{<t})$.
RNN/LSTM limitation	Sequential hidden-state bottleneck and limited parallelism.
Self-attention	Queries, keys, and values create contextual token representations.
Multi-head attention	Different heads capture different relations.
Causal masking	Prevents future-token leakage in decoder-only models.

Exam topic	What you must be able to say
BPE tokenization	Subword units balance vocabulary size and sequence length.

2. Zero-prior explanation: understand the

idea before the equations This section assumes you have not read the lecture yet. The goal is to build intuition first, then connect the intuition to the equations and exam language.

Autoregressive modeling

A sequence probability is written as a product of next-token probabilities. This is both a probabilistic model and a training strategy: every position becomes a supervised next-token prediction.

RNNs and LSTMs

RNNs process tokens sequentially and compress the past into a hidden state. LSTMs improve memory with gates, but they still limit parallelism and can struggle with very long dependencies.

Self-attention from zero prior

Each token creates a query, key, and value. The query asks what the token needs. Keys advertise what tokens contain. Values carry the information to retrieve. Attention weights decide how much to pull from each value.

Memory line

A Transformer is a content-based retrieval machine stacked many times.

3. Core concepts and worked explanations

Transformer block

A standard block combines multi-head attention, a feed-forward network, residual connections, and layer normalization. Attention mixes across tokens; the feed-forward network transforms each token representation.

Position and causal structure

Attention alone has no sense of order. Positional embeddings provide order, and causal masking prevents a decoder-only model from seeing future tokens during training.

Tokenization and GPT training

BPE splits text into subword tokens. GPT-style models train by shifting tokens: each prefix predicts the next token.

4. Method explanation, worked example,

and application

Step	Method explanation
Tokenize	Convert text into subword IDs.
Add position	Inject order information.
Apply causal attention	Each token attends only to earlier tokens.
Predict next token	Use cross-entropy over vocabulary.

Worked example

Example: for the prefix "the cat sat on", the model outputs a probability distribution over possible next tokens such as "the", "a", or "mat". Causal masking prevents the model from seeing the true next token while learning.

Applications and usage

Usage: Transformers power language models, code generation, translation, summarization, chat assistants, VLM decoders, and multimodal systems.

5. Formula / definition cheat sheet

Read formulas as compact sentences. Do not memorize symbols blindly; use the symbol key table after the formula table to understand what each part represents.

Formula	Interpretation
$p(x_{1:T}) = \prod_{t=1}^T p(x_t x_{<t})$ Autoregressive factorization.	$Q = XW_Q$, $K = XW_K$, $V = XW_V$ Attention projections.
$A = \text{softmax}(QK^T / \sqrt{d_k})$ Attention weights.	$\text{Attention}(Q, K, V) = AV$ Weighted value aggregation.
$\text{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2$ Position-wise feed-forward network.	$L = -\sum_t \log p_\theta(x_t x_{<t})$ Next-token cross-entropy.

6. Equation symbol key table

This table explains the main symbols used in the equations. When studying, cover the meaning column and check whether you can explain each symbol clearly.

Symbol	Meaning	Plain-English reading
$x_{1:T}$	Token sequence	Full sequence of tokens.
$x_{<t}$	Prefix	Tokens before position t .
Q	Queries	What each token is looking for.
K	Keys	What each token offers for matching.
V	Values	Content retrieved by attention.
d_k	Key/query dimension	Scaling factor stabilizes dot products.
A	Attention matrix	Weights over tokens.
θ	Model parameters	Learned Transformer weights.

7. Main code prompt / implementation

skeleton This is not meant to replace the lecture code. It is a compact prompt showing the computational pattern you should recognize in homework and exams.

GPT-style training prompt

```
input_ids = tokens[:, :-1] target_ids = tokens[:, 1:]  
logits = transformer(input_ids) # [batch, time, vocab]  
loss = cross_entropy(logits, target_ids)  
loss.backward()  
optimizer.step()
```

How to explain the code in words

Start by identifying the input, the model output, the objective being minimized, and the update rule. Then connect the code back to the formulas and the method table.

8. Common exam traps

Trap	Correct way to think
Saying attention is just	averaging Attention is learned retrieval using Q/K matching and V aggregation.
Forgetting positions	Attention alone does not know token order.
Letting GPT see future tokens	Causal masking is required.

9. Exam-style sample questions with

answers These are written in the style of conceptual exam answers. Use them to practice explaining, not only recognizing terms.

Question 1

Why is autoregressive factorization practical?

Sample answer

It turns sequence learning into next-token prediction at every position.

Question 2

Why use separate Q, K, and V?

Sample answer

They separate asking, matching, and content retrieval roles.

Question 3

Why are residual connections and LayerNorm important?

Sample answer
They stabilize deep training and preserve information flow.

Question 4

Why is causal masking required?

Sample answer
Without it, the model could cheat by attending to future tokens.

Question 5

Why does tokenization matter?

Sample answer
It changes sequence length, vocabulary, rare-word handling, and learned patterns.

10. Flashcards for rapid review

Use these for active recall. Cover the answer column and try to answer before checking.

Card 1
Q: What is autoregressive modeling?
A: Predict each token from previous tokens.

Card 2
Q: What is Q?
A: Query vector.

Card 3
Q: What is K?
A: Key vector.

Card 4
Q: What is V?
A: Value vector.

Card 5
Q: What does softmax do in attention?
A: Normalizes scores into weights.

Card 6
Q: What is causal masking?
A: Blocking future positions.

Card 7
Q: What is BPE?
A: Subword tokenization.

Card 8

Q: What is cross-entropy used for?

A: Next-token training loss.

Card 9

Q: What must you remember about Autoregressive factorization?

A: $p(x_{1:T}) = \prod_t p(x_t | x_{<t})$.

Card 10

Q: What must you remember about RNN/LSTM limitation?

A: Sequential hidden-state bottleneck and limited parallelism.

Card 11

Q: What must you remember about Self-attention?

A: Queries, keys, and values create contextual token representations.

Card 12

Q: What must you remember about Multi-head attention?

A: Different heads capture different relations.

Card 13

Q: What must you remember about Causal masking?

A: Prevents future-token leakage in decoder-only models.

Card 14

Q: What must you remember about BPE tokenization?

A: Subword units balance vocabulary size and sequence length.

Card 15

Q: What does this formula represent:

$p(x_{1:T}) = \prod_{t=1}^T p(x_t | x_{<t})$?

A: Autoregressive factorization.

Card 16

Q: What does this formula represent: $Q = XW_Q$, $K = XW_K$, $V = XW_V$?

A: Attention projections.

Card 17

Q: What does this formula represent:

$A = \text{softmax}(QK^T / \sqrt{d_k})$?

A: Attention weights.

Card 18

Q: What does this formula represent:

$\text{Attention}(Q, K, V) = AV$?

A: Weighted value aggregation.

Card 19

Q: What does this formula represent: $\text{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2$?

A: Position-wise feed-forward network.

Card 20

Q: What does this formula represent: $L = -\sum_t \log p_\theta(x_t | x_{<t})$?

A: Next-token cross-entropy.

Card 21

Q: What does $x_{1:T}$ mean?

A: Token sequence. Full sequence of tokens.

Card 22

Q: What does $x_{<t}$ mean?

A: Prefix. Tokens before position t .

Card 23

Q: What does Q mean?

A: Queries. What each token is looking for.

Card 24

Q: What does K mean?

A: Keys. What each token offers for matching.

Card 25

Q: What does V mean?

A: Values. Content retrieved by attention.

Card 26

Q: What does d_k mean?

A: Key/query dimension. Scaling factor stabilizes dot products.

Card 27

Q: What does A mean?

A: Attention matrix. Weights over tokens.

Card 28

Q: What does θ mean?

A: Model parameters. Learned Transformer weights.

Card 29

Q: What is the key idea of Autoregressive modeling?

A: A sequence probability is written as a product of next-token probabilities. This is both a probabilistic model and a training strategy: every position becomes a supervised next-token

Card 30

Q: What is the key idea of RNNs and LSTMs?

A: RNNs process tokens sequentially and compress the past into a hidden state. LSTMs improve memory with gates, but they still limit parallelism and can struggle with very long

Card 31

Q: What is the key idea of Self-attention from zero prior?

A: Each token creates a query, key, and value.

The query asks what the token needs. Keys advertise what tokens contain. Values carry the information to retrieve. Attention weights decide how much to pull from each

Card 32

Q: What is the key idea of Transformer block?

A: A standard block combines multi-head attention, a feed-forward network, residual connections, and layer normalization. Attention mixes across tokens; the feed-forward network transforms each token

Card 33

Q: What is the key idea of Position and causal structure?

A: Attention alone has no sense of order.

Positional embeddings provide order, and causal masking prevents a decoder-only model from seeing future tokens during

Card 34

Q: What is the key idea of Tokenization and GPT training?

A: BPE splits text into subword tokens. GPT-style models train by shifting tokens: each prefix predicts the next

Card 35

Q: What happens in the step: Tokenize?

A: Convert text into subword IDs.

Card 36

Q: What happens in the step: Add position?

A: Inject order information.

Card 37

Q: What happens in the step: Apply causal attention?

A: Each token attends only to earlier tokens.

Card 38

Q: What happens in the step: Predict next token?

A: Use cross-entropy over vocabulary.

Card 39

Q: Can you write the sequence factorization?

A: Product of next-token conditionals.

Card 40

Q: Can you explain Q/K/V?

A: Ask, match, retrieve.

Card 41

Q: Can you explain why positions matter?

A: Attention alone is permutation-invariant.

Card 42

Q: Can you explain GPT training?

A: Shift inputs and targets by one token.

Card 43

Q: Why is autoregressive factorization practical?

A: It turns sequence learning into next-token prediction at every position.

Card 44

Q: Why use separate Q, K, and V?

A: They separate asking, matching, and content retrieval roles.

Card 45

Q: Why are residual connections and LayerNorm important?

A: They stabilize deep training and preserve information flow.

Card 46

Q: Why is causal masking required?

A: Without it, the model could cheat by attending to future tokens.

Card 47

Q: Why does tokenization matter?

A: It changes sequence length, vocabulary, rare-word handling, and learned patterns.

Card 48

Q: In plain English, why is $p(x_{1:T}) = \prod_{t=1}^T p(x_t | x_{<t})$ important?

A: It captures: Autoregressive factorization. Use it as a compact exam sentence, then define each symbol.

Card 49

Q: In plain English, why is $Q=XW_Q$, $K=XW_K$, $V=XW_V$ important?

A: It captures: Attention projections. Use it as a compact exam sentence, then define each symbol.

Card 50

Q: In plain English, why is $A = \text{softmax}(QK^T / \sqrt{d_k})$ important?

A: It captures: Attention weights. Use it as a compact exam sentence, then define each symbol.

Card 51

Q: In plain English, why is $\text{Attention}(Q, K, V) = AV$ important?

A: It captures: Weighted value aggregation. Use it as a compact exam sentence, then define each symbol.

Card 52

Q: In plain English, why is $\text{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2$ important?

A: It captures: Position-wise feed-forward network. Use it as a compact exam sentence, then define each symbol.

Card 53

Q: In plain English, why is $L = -\sum_t \log p_\theta(x_t | x_{<t})$ important?

A: It captures: Next-token cross-entropy. Use it as a compact exam sentence, then define each symbol.

Card 54

Q: In plain English, why is $p(x_{1:T}) = \prod_{t=1}^T p(x_t | x_{<t})$ important?

A: It captures: Autoregressive factorization. Use it as a compact exam sentence, then define each symbol.

Card 55

Q: In plain English, why is $Q=XW_Q$, $K=XW_K$, $V=XW_V$ important?

A: It captures: Attention projections. Use it as a compact exam sentence, then define each symbol.

Explanation

Chapter 6: Transformers

How this section is organized

Each item below is a heading taken from the corresponding Jupyter notebook in the same order it appears there. The explanation beside it tells you what that notebook section is trying to teach and why it matters for the review.

Source notebook

Chapter_6_Transformers.ipynb

Headings included

34

Notebook headings with explanation

1. Chapter 6 Auto Regressive models and Transformers	Understand what is an auto-regressive model 2. Understand the concept of LSTM and its limitations 4.
2. Autoregressive Models	An autoregressive (AR) model defines a probability distribution over a sequence by factorizing it into a product of next-step conditionals. The autoregressive factorization is: $p(x_{1:T}) = \prod_{t=1}^T p(x_t \text{ mid } x_{1:t-1})$.
3. Simple Autoregressive Example: Predicting the Next Digit (0/1) in a Pattern	We build a tiny autoregressive (AR) model over binary digits. The autoregressive factorization is: $p(x_{1:T}) = \prod_{t=1}^T p(x_t \text{ mid } x_{1:t-1})$.
4. Recurrent Neural Networks (RNNs)	Each step updates the state and predicts the next token. Here A is a neural network which is trained as each token X_i that passes through it in a sequence.
5. A Tiny RNN Autoregressive Model on Binary Strings (0/1)	We build an autoregressive RNN that predicts the next digit in a binary sequence: $x_{1:T}$, x_t in 0,1. We will keep everything small: - Vocabulary size: $V = 2$ (tokens: 0 and 1) - Embedding size: $d = 2$ - Hidden size: $H = 2$ Model definition (parameters and shapes) Embedding matrix We use a learnable embedding matrix: E in $R^{V \text{ times } d}$ - $d = R^{2 \text{ times } 2}$.
6. Long Short-Term Memory (LSTM)	The LSTM introduces a dedicated memory cell c_t and gates that learn when to: - forget old information - write new information - expose (read out) information to make predictions The key intuition is that the cell state c_t (top arrow in the diagram above) can behave like a nearly linear "information highway" through time, while the gates learn *how much* to modify it. What is learned (parameters and what each learns) Assume: - input embedding dimension: d - hidden size: H Inputs: - x_t in R^d (embedding).
7. Summary	The U matrices make the gates context-aware: they tell each gate how to use the previous hidden state h_{t-1} to decide write (U_i), keep/erase (U_f), and reveal (U_o).
8. Backpropagation Through Time (BPTT)	For an RNN/LSTM, the hidden state is computed sequentially: $h_t = f_{\theta}(h_{t-1}, x_t)$, $t=1, \dots, T$. For next-token prediction, a common loss is: $L(\theta) = \sum_{t=1}^{T-1} \ell_t(\theta)$, $\ell_t(\theta) = -\log p_{\theta}(x_{t+1} \text{ mid } x_{1:t})$.
9. What is a Transformer?	A Transformer primary role (at least in its internal layers) is to compute a contextualized representation for every token position. Start with a token sequence: $x_1, x_2, \dots, x_T$ After embedding (token + position), we have initial vectors: $h_1^{\wedge}(0), h_2^{\wedge}(0), \dots, h_T^{\wedge}(0)$.

10. Transformers: Architecture	Transformers became the dominant sequence modeling architecture because they address two core bottlenecks of LSTMs: 1. Sequential computation (slow training/inference for long sequences) 2.
11. Transformer: Why we need “Attention”	Consider the sentence: > “the pink elephant tried to get into the car but it was too ...” When you reach the word “too”, you (as a human) immediately know you must look back in the sentence to decide: - too what? too *big*? too *small*? too *old*? - what is “it”? does it refer to the car or the elephant? - what earlier word(s) provide the key information to interpret “too”? The core problem: not all past words matter equally At the moment you process “too”, the most useful tokens are likely: - “car” (it).
12. Transformer Architecture	This notebook heading introduces transformer architecture and explains how it supports the chapter concepts, equations, method steps, or implementation workflow.
13. Understanding “Attention” using the diagram (one attention head)	> Goal: compute a *contextualized representation* of the current token (here the last token “too”) by looking back at all tokens in the sequence and deciding which ones matter most. Start with token embeddings (gray blocks) Each token in the sentence (e.g., “the”, “pink”, “elephant”, ..., “too”) is mapped to an embedding vector: - Embedding dimension: d_e - Token embedding for position j : x_j in $\mathbb{R}^{d_e}$ In the diagram, the gray bars are these vectors x_j each representing an embedding for a token.
14. Multihead Attention	Each head has its own learned projections, so different heads can specialize in different relations (e.g., local syntax, long-range coreference, etc.). Setup and shapes Let the input to attention be: X in $\mathbb{R}^{T \times d}$, where: - T = sequence length - d = model dimension Choose number of heads H , and per-head dimensions: $d_k = d_v = d/H$. (So concatenating all heads returns to dimension d .) One head (head h) Each head has its own projection matrices: $W_Q^{(h)}$ in $\mathbb{R}^{d \times d_k}$, $W_K^{(h)}$ in $\mathbb{R}^{d \times d_k}$.
15. Why multiple heads help	A single attention head produces one set of weights per token: it can focus on only one “pattern” at a time. - Multiple heads allow the model to compute multiple different attention patterns in parallel. - After concatenation, the model keeps all these perspectives and can combine them with W_O . Summary Multi-head attention runs several attention mechanisms in parallel, each with its own learned projections, then concatenates their outputs to produce a richer contextual representation.
16. Example: A 3-token walk-through of self-attention	From the earlier sentence (and picture), we consider only three tokens: 1. elephant 2. car 3. too (this is the token whose context we want to compute) In the picture: - Query (green) comes from the current token (“too”) - Keys (red) come from *every token* in the sequence - Values (yellow) come from *every token* in the sequence - Softmax weights are the attention weights shown next to tokens - The output is the weighted sum of value vectors (orange) Step 0 — Embeddings (gray blocks) Let the embedding.
17. Matrix Form Summary of a Transformer	Assume a sequence of length T with token ids: $s = (s_1, \dots, s_T)$, s_t in $1, \dots, V$. Let model dimension be d , head key dimension d_k , and value dimension d_v .
18. Layer Normalization	LayerNorm normalizes each token’s feature vector across its hidden dimensions, then applies a learned scale and shift. It is used heavily in Transformers to stabilize training and improve gradient flow.
19. The Transformer Block	Row x_t is token t ’s vector. - Output: a new matrix X_{out} in $\mathbb{R}^{T \times d}$ same shape as X , but each token vector is more contextual. Multi-head attention layer (orange) Input - Input: the current sequence representations X in $\mathbb{R}^{T \times d}$.
20. Simple Implementation of a Transformer Block	Now we implement a full-fledged implementation of a Transformer. Matplotlib is optional (used for a small visualization). The notebook also includes 3 nearby code cells for implementation, demonstration, or visualization of this heading.

21. Byte-Pair Encoding (BPE) Tokenization	Modern language models do not operate directly on raw characters or raw words. Instead, they convert text into tokens drawn from a fixed vocabulary.
22. GPT-2 uses byte-level BPE:	Below is a compact, educational byte-level BPE implementation. Train BPE on the corpus BPE must be trained on a specific corpus of text. The notebook also includes 4 nearby code cells for implementation, demonstration, or visualization of this heading.
23. Attention: A Simple Example	Single-head attention is: $\text{Att}(Q,K,V) = \text{softmax}(\frac{QK^{\text{top}}}{\sqrt{d_k}})V$. Causal masking prevents position t from attending to future positions $>t$. The notebook also includes 3 nearby code cells for implementation, demonstration, or visualization of this heading.
24. GELU MLP in a Transformer (FFN)	Inside each Transformer block, the MLP / Feed-Forward Network (FFN) is a position-wise network applied independently to each token vector. It adds nonlinearity and capacity after attention has mixed information across tokens.
25. How training works with GPT-style models	A GPT-style language model is trained to do next-token prediction. Suppose you have a tokenized sequence: $s_1, s_2, \dots, s_T, s_{t+1}$ in $1, \dots, V$.
26. A small GPT-2 style Decoder-only Transformer	This implements: - Token embeddings W_e and learned positional embeddings W_p - Pre-LN Transformer blocks with masked multi-head self-attention - GELU MLP - Weight tying between embedding and LM head The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
27. Prepare tokenized dataset and batching	This notebook heading introduces prepare tokenized dataset and batching and explains how it supports the chapter concepts, equations, method steps, or implementation workflow. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
28. Train the model	This notebook heading introduces train the model and explains how it supports the chapter concepts, equations, method steps, or implementation workflow. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
29. Generate text	This notebook heading introduces generate text and explains how it supports the chapter concepts, equations, method steps, or implementation workflow. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
30. Inspect next-token probabilities	This notebook heading introduces inspect next-token probabilities and explains how it supports the chapter concepts, equations, method steps, or implementation workflow. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
31. Common Autoregressive (next-token) foundation models	> Autoregressive means the model is trained to predict the next token: $p(x_{1:T}) = \prod_{t=1}^T p(x_t x_{1:t-1})$. Most modern LLM chat systems are deployed as decoder-style Transformer generators (often with MoE, tools, and multimodal input). Provider; Model (representative); Open weights?; Encoder/Decoder style; Context window (typical); Modalities; Notes / distinguishing traits OpenAI; GPT-5 (plus GPT-5 mini / nano); No; Decoder-style Transformer (AR); 400K context, 128K max output; Text + vision (image input);.
32. Sources (links)	OpenAI - https://openai.com/gpt-5/ - https://developers.openai.com/api/docs/models/gpt-5 - https://developers.openai.com/api/docs/models/gpt-5-mini - https://developers.openai.com/api/docs/models/gpt-5-nano Anthropic (Claude 4.6) - https://platform.claude.com/docs/en/about-claude/models/whats-new-claude-4-6 - https://www.anthropic.com/news/claude-opus-4-6 - https://www.anthropic.com/news/claude-sonnet-4-6 DeepSeek - https://api-docs.deepseek.com/quick_start/pricing -.

33. Other Transformer architectures: decoder-only vs encoder-only vs encoder-decoder	Architecture; Core idea; Attention pattern; Typical training objective; Strengths; Common uses; Representative model families Decoder-only (causal LM); Generate next token from the prefix; Causal self-attention (token t attends only to $\leq t$); Autoregressive next-token prediction; Best for open-ended generation; scales well; natural for chat/code; Chatbots, text generation, code generation, long-form writing, agentic tool use; GPT (OpenAI), Claude (Anthropic), Gemini (Google; generative), Llama (Meta),.
34. Objectives (equations)	Decoder-only (autoregressive) objective Given tokens $s_{1:T}$: $p_{\theta}(s_{1:T}) = \prod_{t=1}^T p_{\theta}(s_t \mid s_{<t})$, and the negative log-likelihood loss: $L(\theta) = -\sum_{t=1}^T \log p_{\theta}(s_t \mid s_{<t})$. Encoder-only (masked language modeling) Let M be the set of masked positions.